CS 2010 Lab 07 — Handout

Gate symbols, DNF and CNF

2026-10-19

1. Notation

Throughout the lab, boolean formulas are written in the short algebraic notation:

Write	Means
`ab`	`a` AND `b` (letters side by side)
`a + b`	`a` OR `b`
`a'`	NOT `a`
`( ... )`	group first

So `(ab) + (a+b')'` is read as:

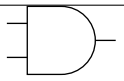
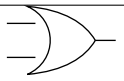
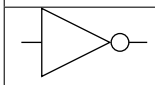
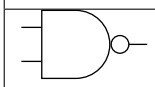
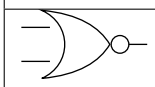
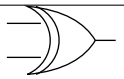
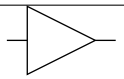
> (`a` AND `b`) OR NOT (`a` OR NOT `b`)

Two conventions worth stating once:

- **`'` binds tightest**, then AND, then OR. `ab'` means `a` AND `(NOT b)`, and `a + bc` means `a` OR (`b` AND `c`).
- **Multi-input gates are allowed on paper.** You may draw a 3-input AND gate. The Luanti blocks are all 2-input, so a 3-input AND is built by chaining two 2-input ANDs.

2. Gate symbols

Each logic gate has a standard shape, called its **distinctive shape**. You draw the shape, not the name.

Symbol	Name	Expression	Output is 1 when ...
	AND	ab	both inputs are 1
	OR	$a + b$	at least one input is 1
	NOT	a'	the input is 0
	NAND	$(ab)'$	not both inputs are 1
	NOR	$(a + b)'$	both inputs are 0
	XOR	$ab' + a'b$	the inputs differ
	buffer	a	the input is 1 (passes the signal through)

Notes:

- **Inputs go on the left, the output on the right.** The short stubs on each symbol above are where the wires connect.
- A **small circle** on the output means NOT. That is the only difference between AND/NAND, OR/NOR, and buffer/NOT.
- The **buffer** (also called a diode) computes nothing. It exists to let a signal travel one way. You will not need it unless a wire is awkward.
- In Luanti, the mesecons blocks are labelled with these same shapes, so the sheet above is also your block identification guide.
- The 2-input gates take one input on each side; the output faces the direction the gate is turned. Ask if a gate seems to ignore one input.

3. Truth tables

With n inputs there are 2^n rows. Write the input columns in counting order, so you never miss a row:

```
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```

That is the order used everywhere in this lab.

4. DNF — disjunctive normal form

A formula is in DNF when it is an OR of AND-terms, where each term is variables and negated variables ANDed together. For example `a'bc + ab'c` is in DNF; `(a+b)c` is not.

How to get the DNF of a function from its truth table:

> For **every row where the output is 1**, write one AND-term: take each input, un-negated if it is 1 in that row and negated if it is 0. Then OR all of those terms together.

Worked example — XOR, `F = ab' + a'b`:

a	b	F
0	0	0
0	1	1
1	0	1
1	1	0

Rows with `F = 1` are `01` and `10`.

- Row `01`: `a` is 0 so it becomes `a'`; `b` is 1 so it stays `b`. Term: `a'b`.
- Row `10`: term `ab'`.

DNF: `a'b + ab'`. Two 1-rows, two terms.

5. CNF — conjunctive normal form

CNF is the mirror image. **A formula is in CNF when it is an AND of OR-clauses**, where each clause is variables and negated variables ORed together. For example `(a+b)(a'+b')` is in CNF.

How to get the CNF of a function from its truth table:

> For **every row where the output is 0**, write one OR-clause: take each input, un-negated if it is 0 in that row and negated if it is 1. Then AND all of those clauses together.

The flip is the whole trick: a clause is written so that it is **false on exactly its own row** and true everywhere else. Check it: the clause for row `11` is `(a' + b')`, which is false only when `a=1` and `b=1`.

Worked example — the same XOR function:

Rows with `F = 0` are `00` and `11`.

- Row `00`: both inputs are 0, so both stay un-negated. Clause: `(a + b)`.
- Row `11`: both inputs are 1, so both are negated. Clause: `(a' + b')`.

CNF: `(a + b)(a' + b').` Two 0-rows, two clauses.

Both forms describe the same function. They are just built from opposite rows.

6. Which form is shorter?

- **DNF gets big when the output is 1 a lot** — one term per 1-row.
- **CNF gets big when the output is 0 a lot** — one clause per 0-row.

So for a function that is 1 on almost every row, DNF is enormous and CNF is tiny. Part 4 of the lab asks you to see exactly that, on four inputs.

Neither form is "the simplified one". Both are *normal* forms: a fixed recipe that always works, and always gives you a circuit exactly two levels deep (one level of ANDs, one level of ORs). A normal form is often **not** the smallest circuit — it is the most predictable one.

7. Truth table for Part 4

Fill in the **F** column. The function is: *the light is ON unless all four switches are in the same position.* In other words `F = 1` when the four inputs are not all equal.

a	**b**	**c**	**d**	**F**
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	